

Automated Electricity Demand Forecasting and Intelligent Model Selection Platform



IN ACADEMIC PARTNERSHIP WITH



Author: Paras Rimal

Student ID: 25123819

Lecturer: Umesh Regmi

Moodle Title: Django and Fast API

Repository: <https://github.com/ParasRimal/GridDemand-Forecasting-AI>

Abstract

Accurate day-ahead forecasts of electricity demand support the planning of generation and purchases, but a forecasting model is only useful if it is trustworthy, monitored and replaced only when something better is proven. This report describes GridPredict, an end-to-end system that forecasts the hourly electric load of Ahmedabad, India, 24 hours ahead using one year of hourly weather and load data (8,779 hours, November 2021 to November 2022).

The report documents the data analysis, the cleaning rules (24 invalid irradiance values and 79 load dropouts set to missing, never interpolated), and a leakage-safe feature pipeline with calendar, weather, lag (24, 48, 168 hours) and rolling features, giving 8,532 modelling rows. Random Forest, XGBoost and LightGBM are compared against naive baselines on a chronological split and by a seven-fold rolling-origin backtest. On validation, XGBoost beat the "same hour yesterday" baseline (MAE 8.47 against 9.25 MW), and a Random Forest without season columns won the backtest (mean MAE 8.44 MW, 6 of 7 folds). However, on the held-out test period both lost to the baseline (10.67 and 10.40 MW against 9.01 MW). An error analysis shows that the Random Forest is slightly better in hours up to 100 MW but much worse in high-load hours, which is consistent with the failure of tree ensembles to follow a rising load level.

A promotion gate requiring an improvement of more than 0.3 MW rejected both challengers, so the naive baseline remains in production. A drift check flags 14 of 19 features (73.7 %), mainly because of seasonal change, while the forecast error is unchanged. The system is served by FastAPI with a fail-open Redis cache, a Django REST backend, a daily Celery monitoring job and a React dashboard, packaged in six Docker services and covered by 187 automated test functions. The main contribution is a trustworthy process that detects and reports a negative result rather than deploying a worse model.

Keywords: short-term load forecasting; Random Forest; XGBoost; LightGBM; rolling-origin backtest; data leakage; model registry; drift monitoring; FastAPI; Django; React

Table of Contents

Table of Contents.....	2
Table of Figures.....	4
1. Introduction.....	5
1.1 Background of the Project.....	5
1.2 Problem Statement.....	5
1.3 Purpose of the Project.....	6
2. Objectives.....	7
3. Methodology.....	9
3.1 Overall Methodology.....	9
3.2 Dataset Collection and Screening.....	9
Dataset 1: Power Load Dataset (rejected).....	9
Dataset 2: Weather and Load Dataset, Ahmedabad (selected).....	10
3.3 Data Preprocessing.....	12
3.4 Feature Engineering.....	12
Time-based (calendar) features — 7.....	13
Weather features — 5.....	13
Lag features — 3.....	13
Rolling features — 4.....	13
Leakage verification.....	13
3.5 Machine Learning Models.....	13
3.6 Model Evaluation and Selection.....	14
Evaluation metrics.....	14
The naive baseline.....	14
Why model selection was rebuilt during the project.....	15
3.7 Automated Monitoring and Retraining Decision.....	15
3.8 System Development Technologies.....	15
4. Project Implementation.....	17
4.1 System Architecture.....	17
4.2 Data Processing Pipeline.....	17
4.3 Machine Learning Pipeline.....	18
4.4 Django Backend.....	18
4.5 FastAPI Machine Learning Service.....	19
4.6 React Frontend.....	20
4.7 SQLite Database.....	23
4.8 Redis Cache and Message Broker.....	23
4.9 Docker Containerisation.....	23
4.10 Automated Monitoring and Retraining Decision Pipeline.....	24
5. Results and Output.....	25
5.1 Validation Results.....	25
5.2 Test Set Results — The Headline Finding.....	25
5.3 Rolling Backtest and Re-Selection.....	26
5.4 The Re-Selected Candidate, Reported Honestly on the Test Set.....	27

5.5 Promotion Gate Decisions.....	28
5.6 Feature Importance.....	28
5.7 Drift Monitoring Result.....	29
5.8 Testing Summary.....	29
5.9 Summary of Outcomes.....	30
6. Challenges and Limitations.....	31
6.1 Dataset Compatibility.....	31
6.2 Time-Series Data Leakage.....	31
6.3 Changing Data Patterns.....	31
6.4 Limited Dataset Size.....	31
6.5 Real-Time Data Availability.....	31
6.6 Computational Resources.....	31
6.7 Additional Engineering Challenges Encountered.....	32
7. Conclusion.....	33
8. Future Scope.....	34
8.1 Real-Time Data Integration.....	34
8.2 Longer Forecast Horizons.....	34
8.3 A Level-Aware or Hybrid Model.....	34
8.4 Deep Learning and Probabilistic Forecasting.....	34
8.5 Geographic Forecasting and Cloud Deployment.....	34
8.6 Full MLOps Tooling.....	34
9. References.....	35
10. Appendices.....	37
Appendix A: Selected Datasets.....	37
Appendix B: Implemented Frontend Pages.....	37
Appendix C: Repository and How to Run the System.....	37
Appendix D: Automated Test Summary.....	38

Table of Figures

- Fig. 1: Dataset screening — PowerLoad_Dataset.csv shows no daily pattern, while the Ahmedabad dataset shows a clear one.
- Fig. 2: Average electricity load by hour of day (Ahmedabad dataset).
- Fig. 3: Average electricity load by day of week (Ahmedabad dataset).
- Fig. 4: Average electricity load by month, Nov 2021 – Nov 2022 (Ahmedabad dataset).
- Fig. 5: Correlation between weather variables and load (Ahmedabad dataset).
- Fig. 6: Autocorrelation of load at selected lags (Ahmedabad dataset).
- Fig. 7: GridPredict system architecture, as implemented.
- Fig. 8: GridPredict machine learning and promotion pipeline, as implemented.
- Fig. 9: Django administration, Model versions list view.
- Fig. 10: FastAPI interactive documentation, listing the five service endpoints.
- Fig. 11: Dashboard page — production model, latest drift check, and model comparison history.
- Fig. 12: Try a Forecast page — a real prediction returned by the FastAPI service.
- Fig. 13: Model History page — MAE comparison chart and the full promotion decision log.
- Fig. 14: System Status page — live health of Django, FastAPI, and Redis.
- Fig. 15: Prediction request flow with Redis caching, as implemented.
- Fig. 16: Docker Compose service composition, as implemented.
- Fig. 17: Automated monitoring and retraining decision pipeline, as implemented.
- Fig. 18: Validation set results, July–August 2022 ($n = 1,448$).
- Fig. 19: Test set results, September–November 2022 ($n = 1,481$).
- Fig. 20: XGBoost vs the baseline, fold by fold, across the 7-fold rolling backtest.
- Fig. 21: Rolling backtest summary across all evaluated candidates.
- Fig. 22: Feature importance of the saved Random Forest candidate model.
- Fig. 23: Feature drift between the training period and the test period.

1. Introduction

1.1 Background of the Project

Electricity is one of the most important resources required by modern society, and electricity demand changes continuously depending on factors such as time of day, day of the week, weather conditions, holidays, human activity, and seasonal patterns. Accurate electricity demand forecasting is therefore important for efficient power generation, distribution, resource planning, and grid management.

If electricity demand can be predicted accurately in advance, energy providers can better prepare for periods of high or low demand, reduce unnecessary generation, improve resource utilisation, and support more reliable distribution. Traditional forecasting approaches may not capture the complex, non-linear relationships between consumption, weather, and temporal patterns, whereas machine learning can learn these relationships directly from historical data.

This report documents GridPredict, a complete, working implementation of an automated electricity demand forecasting platform that combines machine learning with modern web technologies and MLOps practice. The system processes historical electricity and weather data, trains multiple machine learning models, evaluates their performance using a methodology designed specifically to avoid the common pitfalls of time-series forecasting, automatically selects (or withholds) a production model, and serves predictions through a web dashboard and two independent APIs.

Unlike the proposal stage of this project, every number, chart, and screen described below is drawn from the finished, deployed system rather than from a plan of what was intended. Where the system behaved differently from the original expectation, that outcome is reported honestly rather than adjusted to look more favourable, because an accurate account of what a forecasting system actually does is more valuable, and more representative of real MLOps practice, than a report that only shows success.

1.2 Problem Statement

Electricity demand is dynamic and influenced by multiple variables. A forecasting model trained on historical data may gradually become less accurate as the characteristics of incoming data change. Changes in weather conditions, demand patterns, seasonal behaviour, and other external factors can degrade the performance of a deployed machine learning model. Many introductory electricity-forecasting projects stop at training a model and producing a prediction; they do not provide an integrated system for model monitoring, automated retraining, model comparison, API-based prediction, caching, or visualisation, and critically, they rarely verify that a trained model is actually better than the simplest possible alternative before deploying it.

GridPredict was built to provide an integrated electricity demand forecasting system that:

- Processes historical electricity and weather data.
- Identifies temporal and environmental patterns in that data.
- Trains multiple machine learning models.
- Compares model performance using appropriate evaluation metrics and a methodology resistant to data leakage.
- Automatically selects the best-performing model, and only promotes it if it is demonstrably better than what is already in production.
- Monitors incoming data for significant statistical change (drift).

- Triggers a retraining recommendation when drift or performance degradation is detected.
- Exposes machine learning predictions through a REST API.
- Presents predictions, historical patterns, model performance, and monitoring information through an interactive web dashboard.
- Runs as a set of containerised services that can be started and stopped as a single unit.

1.3 Purpose of the Project

The purpose of GridPredict is to demonstrate, in a fully working system rather than a notebook, how a forecasting model is developed, evaluated, deployed, monitored, and governed. The project integrates machine learning, backend web development, REST API development, database design, caching, task scheduling, containerisation, and frontend development into one coherent application, and it demonstrates the MLOps discipline of refusing to deploy a model that does not earn its place in production.

2. Objectives

The objectives below are listed as originally defined, with the completion status of each noted against the finished system.

#	Objective	Status
1	Collect and preprocess two electricity and weather-related datasets.	Achieved
2	Analyse historical electricity demand and identify temporal and environmental patterns.	Achieved
3	Perform feature engineering using time-based, weather-based, and historical load features.	Achieved (19 features)
4	Develop multiple machine learning regression models for demand forecasting.	Achieved
5	Implement at least two machine learning models and compare their predictive performance.	Achieved (3 models)
6	Use MAE, RMSE, MAPE, and R^2 to assess model performance.	Achieved
7	Automatically select the best-performing model based on evaluation results.	Achieved, with a promotion gate that can refuse a candidate
8	Develop a FastAPI-based machine learning service for prediction and model information.	Achieved
9	Develop a Django backend for application logic, database management, and REST APIs.	Achieved
10	Develop an interactive React frontend for forecasts, trends, performance, and monitoring.	Achieved (8 pages, exceeding the original plan)
11	Use SQLite for persistent application data storage.	Achieved

#	Objective	Status
12	Use Redis for caching frequently requested forecast and model information.	Achieved, with a fail-open design
13	Implement automated model monitoring and retraining decisions.	Achieved (decision logic implemented; full automatic model-swap-on-drift deferred, see Section 6)
14	Containerise the major components of the system using Docker.	Achieved (6 containers)
15	Deploy the complete application as an integrated system.	Achieved and verified end to end

3. Methodology

3.1 Overall Methodology

The project followed an end-to-end machine learning and software development methodology:

Data Collection → Data Validation → Data Preprocessing → Feature Engineering → Chronological Split → Model Training → Model Evaluation → Backtested Model Selection → Promotion Gate → Deployment → Prediction → Monitoring → Retraining Decision.

The machine learning component was not developed in isolation. The same Python package (`ml_service/app/`) that performs training and evaluation is imported directly by the FastAPI service, the Django backend, and the Celery automation layer, so there is a single source of truth for every threshold, feature, and metric used across the system.

3.2 Dataset Collection and Screening

Two datasets were obtained for this project, and both were inspected in detail before either was committed to as the primary dataset for modelling. This screening step was not part of the original plan but became necessary once the first dataset was examined closely.

Dataset 1: Power Load Dataset (rejected)

`PowerLoad_Dataset.csv` contains 10,000 records and 10 variables: `Timestamp`, `Power_Load_kW`, `Temperature_C`, `Humidity_%`, `WindSpeed_mps`, `Precipitation_mm`, `DayOfWeek`, `HolidayFlag`, `Daily_PostDispatch_Load`, and `Weekly_PreDispatch_Projection`. On inspection, this dataset showed clear evidence of being synthetic rather than measured: timestamps were irregularly spaced (only around one hour in five was present, with the gap between consecutive rows following a geometric distribution typical of independently-dropped rows rather than real sensor gaps), the weather columns were uniformly distributed over suspiciously round ranges, every weather variable correlated with the

load column at under 0.02, and the load showed no detectable daily, weekly, or seasonal pattern and near-zero autocorrelation at every lag tested from 1 to 168 hours. Figure 1 compares the hourly load profile of this dataset against the dataset that was ultimately selected.

Dataset Screening: Hourly Load Pattern Comparison

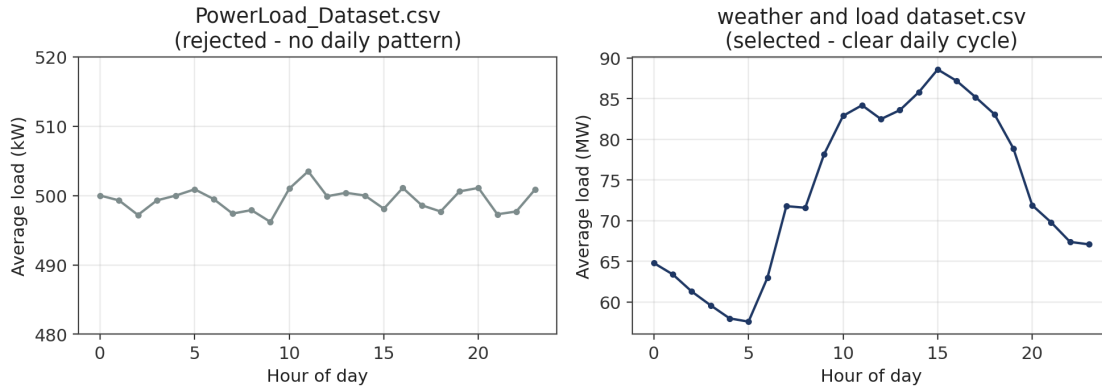


Fig. 1: Dataset screening — *PowerLoad_Dataset.csv* shows no daily pattern, while the *Ahmedabad dataset* shows a clear one.

Because no model can learn a genuine pattern from data that does not contain one, this dataset was excluded from the primary forecasting pipeline. It is retained in the project only as documented evidence for this screening decision.

Dataset 2: Weather and Load Dataset, Ahmedabad (selected)

weather and load dataset.csv contains 8,779 hourly records (1 November 2021 to 1 November 2022) and 10 variables: YEAR, Month, Day, Hour, irradiance, temperature, dewpoint, specific humidity, wind speed, and Electric Load (MW). Unlike the first dataset, this one has a complete hourly timeline with zero missing hours, and clear, learnable structure:

- A daily cycle: average load falls to about 58 MW around 5 a.m. and rises to about 89 MW around 3 p.m.
- A weekly cycle: Sunday is the lowest day and Thursday to Saturday are the highest.
- A seasonal cycle: November to January is lowest, and April to June and September are highest.
- A meaningful correlation with weather: temperature correlates with load at 0.663 and irradiance at 0.366.
- Strong autocorrelation: 0.940 at a 1-hour lag, 0.820 at 24 hours, and 0.665 at 168 hours.

These statistics, all measured directly from the dataset, are presented in Figures 2 to 5.

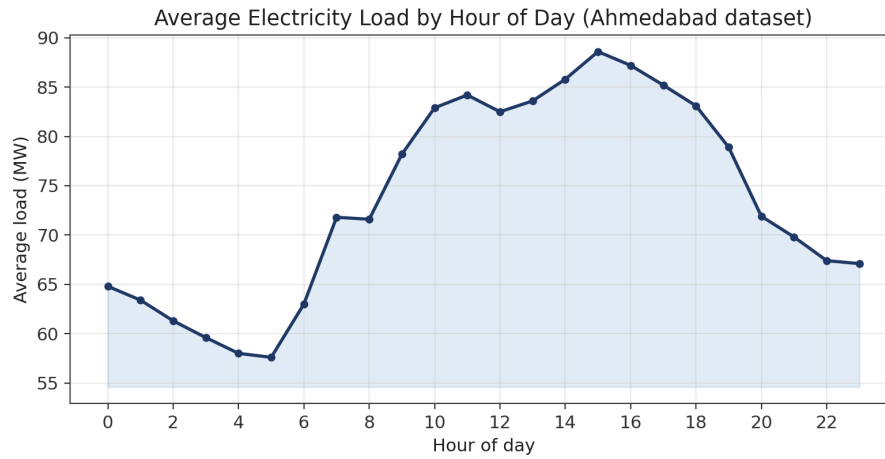


Fig. 2: Average electricity load by hour of day (Ahmedabad dataset).

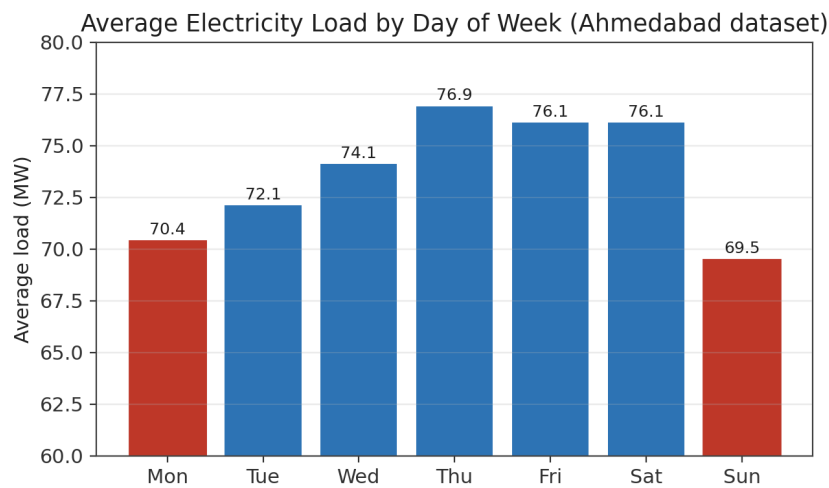


Fig. 3: Average electricity load by day of week (Ahmedabad dataset).

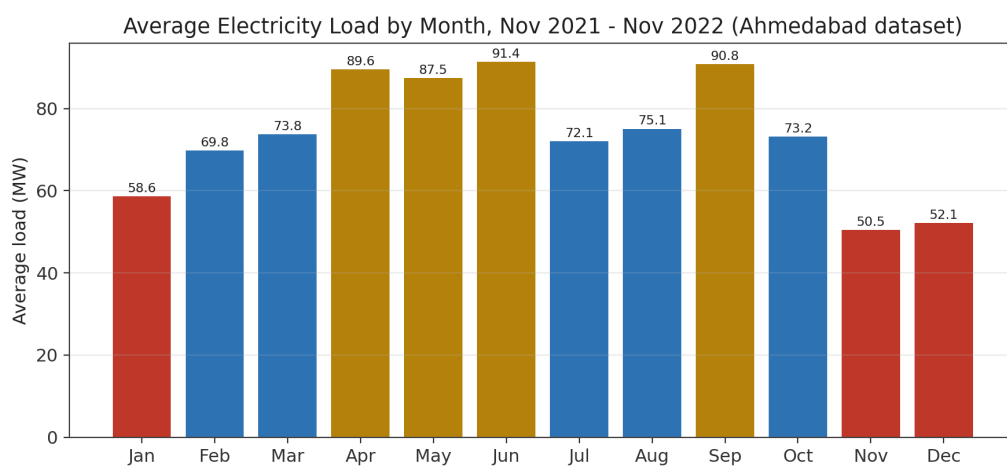


Fig. 4: Average electricity load by month, Nov 2021 – Nov 2022 (Ahmedabad dataset).

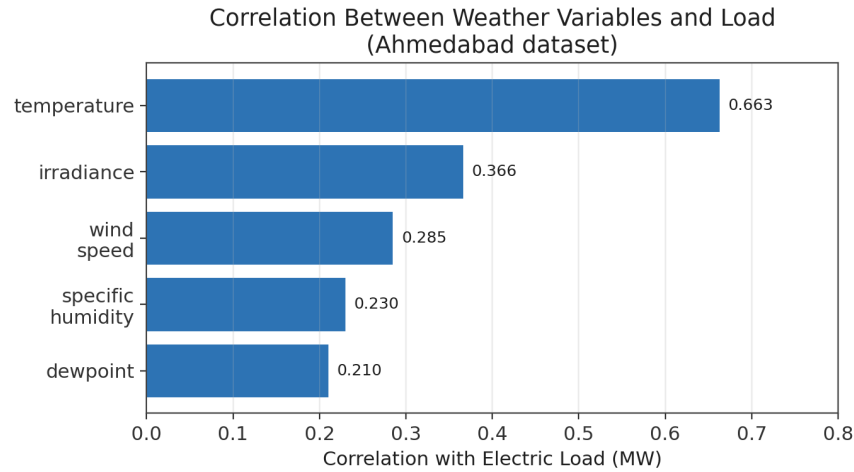


Fig. 5: Correlation between weather variables and load (Ahmedabad dataset).

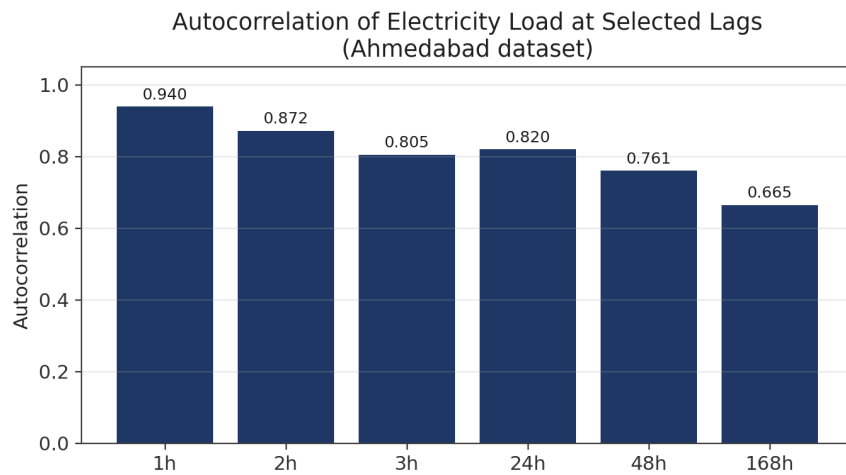


Fig. 6: Autocorrelation of load at selected lags (Ahmedabad dataset).

On the strength of this evidence, the Ahmedabad weather and load dataset was adopted as the sole primary dataset for all subsequent modelling. Because the two source datasets differ in time period, geography, units, and structure, they were never merged, which avoided the risk of incorrect temporal matching and data leakage that the original proposal had already identified as a concern.

3.3 Data Preprocessing

The preprocessing pipeline (`ml_service/app/preprocessing/data_loader.py`) performs the following steps, each covered by an automated test:

- Builds a single Timestamp index from the separate YEAR, Month, Day, and Hour columns.
- Rejects the file outright if a required column is missing or if duplicate timestamps are present.
- Sorts the data chronologically if it is not already sorted.
- Re-indexes the series onto a complete hourly timeline, inserting missing hours as rows of missing values rather than silently dropping them, so that later lag features remain exactly correct.
- Converts irradiance values of exactly `-999` (a sensor “no measurement” code, found as one continuous 24-hour block on 7–8 January 2022) to missing.

- Converts load values of zero or below (79 rows in 22 short runs, almost all exactly 0.0, consistent with meter dropouts) to missing.

At no point are missing values interpolated or filled with invented numbers; a model trained on fabricated values would be learning from data that never happened. Rows with a missing target are excluded only at the very last step of the pipeline, after lag and rolling features have already been computed on the complete timeline.

3.4 Feature Engineering

The system forecasts the load 24 hours ahead of the forecast time. Every feature was designed, and later tested, to use only information that would genuinely be available 24 hours before the hour being predicted. Nineteen features are produced in total, in four groups.

Time-based (calendar) features — 7

Feature	Description
hour	Hour of day (0–23)
day_of_week	Day of week (0 = Monday)
is_weekend	Saturday/Sunday indicator
month	Calendar month
quarter	Calendar quarter
week_of_year	ISO week number
is_holiday	India and Gujarat state holiday indicator

day and year were deliberately excluded: with only one year of data, they would let a tree-based model memorise individual calendar dates rather than learn a generalisable pattern.

Weather features — 5

irradiance, temperature, dewpoint, specific humidity, and wind speed, taken at the target hour. This is a deliberate simplifying assumption, discussed further in Section 6, since a live system would only have a weather forecast rather than the true observed value.

Lag features — 3

lag_24, lag_48, and lag_168: the load exactly 1 day, 2 days, and 1 week before the target hour. Lags are looked up by timestamp, not by row position, so a missing earlier hour correctly produces a missing lag value instead of a silently wrong one. Any lag shorter than the 24-hour forecast horizon is rejected by an assertion in the system’s configuration, because it would use information not yet available at forecast time.

Rolling features — 4

rolling_mean_24, rolling_std_24, rolling_mean_168, and rolling_std_168: the mean and standard deviation of the load over the 24-hour and 168-hour windows ending 24 hours before the target hour, never a window that includes the target hour itself.

Leakage verification

Because electricity load is highly autocorrelated, a single leaked hour of “future” information can make a model look dramatically better than it really is, and this risk is listed explicitly in the original proposal (Section 6.2 of this report). Leakage was therefore treated as a testable property, not an

assumption. An automated end-to-end test deliberately corrupts the load values from 23 hours before the target hour up to the target hour itself and asserts that none of the 19 computed features change as a result. This test, along with 174 others, passes on every run of the test suite (Section 5.5).

3.5 Machine Learning Models

Three regression models were trained, each with fixed, untuned hyperparameters and a fixed random seed (42), so that every comparison in this report is reproducible.

- Random Forest Regressor (Breiman, 2001) — an ensemble of decision trees trained on bootstrapped samples, able to capture non-linear relationships between load and its predictors without assuming a particular functional form.
- XGBoost Regressor (Chen and Guestrin, 2016) — a gradient-boosted tree ensemble that builds trees sequentially, each correcting the errors of the previous ones.
- LightGBM Regressor (Ke et al., 2017) — a second gradient-boosting implementation using histogram-based splitting, included as a third point of comparison.

All three models were implemented using scikit-learn and the vendor libraries' own scikit-learn-compatible interfaces (Pedregosa et al., 2011), and all three were confirmed to handle the missing values produced by the preprocessing pipeline natively, without any imputation step.

3.6 Model Evaluation and Selection

Electricity load is a time series, so a random train/test split would allow the model to be evaluated on hours that occurred before some of its training data, understating the real difficulty of forecasting the future. The data was therefore split chronologically into three contiguous blocks:

Split	Period	Rows	Purpose
Train	8 Nov 2021 – 30 Jun 2022	5,583	Fit the models
Validation	1 Jul 2022 – 31 Aug 2022	1,468	Compare models during development
Test	1 Sep 2022 – 1 Nov 2022	1,481	Final, held-out evaluation

Evaluation metrics

Four metrics were implemented and independently verified against scikit-learn's own implementations on 500 points of random data (Pedregosa et al., 2011):

- Mean Absolute Error (MAE): the average absolute difference between predicted and actual load, in MW. This was treated as the primary metric because it is the most directly interpretable.
- Root Mean Squared Error (RMSE): similar to MAE but penalises large errors more heavily.
- Mean Absolute Percentage Error (MAPE): error expressed as a percentage of actual load (Hyndman and Koehler, 2006, discuss the strengths and well-known weaknesses of this measure).
- R^2 : the proportion of variance in the load explained by the model, where 1.0 is a perfect fit and 0.0 is no better than always predicting the mean.

The naive baseline

Before any machine learning model was trained, a naive baseline was established: predict that the load at a given hour tomorrow will equal the load at the same hour today (the lag_24 feature, used directly

as the prediction). Because electricity demand repeats strongly on a daily cycle (Section 3.2), this simple rule is a genuinely difficult benchmark to beat, and every subsequent model in this project was required to beat it on the same rows before being considered for production.

Why model selection was rebuilt during the project

The first version of the selection process picked whichever model had the lowest MAE on the single two-month validation period. Under that rule, XGBoost was selected. When that model was then scored once on the held-out test period, it performed worse than the naive baseline (Section 5 gives the exact figures). This showed that a single two-month window was not a reliable basis for choosing a production model: the result that looked best in validation did not generalise.

The selection process was therefore rebuilt around a 7-fold rolling-origin backtest. Each fold trains on all data available before a given calendar month (February to August 2022 in turn), leaves a 24-hour gap equal to the forecast horizon, and tests on that month. A model is only eligible for selection if it beats the naive baseline in at least 70% of the seven folds and has a lower mean MAE across all folds; among eligible candidates, ties within 0.1 MW are broken by whichever model beat the baseline in the most folds. This rule was written down before the backtest was run, so that the result could not be chosen after the fact to favour a particular model.

3.7 Automated Monitoring and Retraining Decision

After a model is in production, the statistical properties of incoming data can change in ways that make the model less accurate. GridPredict monitors this directly rather than retraining on a fixed schedule regardless of evidence, in line with the proposal’s original intention not to retrain simply because “new data arrived”.

- **Drift detection:** Evidently (Evidently AI, 2026) compares the statistical distribution of each of the 19 features between a reference period and a current period. Numeric features are compared using the Wasserstein distance and binary features using the Jensen–Shannon distance when the sample is large, consistent with standard practice in monitoring evolving data distributions (Hinder, Vaquet and Hammer, 2024). A dataset is flagged as drifted when more than 50% of its features have individually drifted.
- **Performance monitoring:** the recent mean absolute error is compared against a documented reference MAE; a relative increase of more than 15% is flagged as performance degradation.
- **Combined decision:** retraining is recommended only when drift and/or degradation is actually present, with a written, logged reason — never automatically, and never merely because time has passed or new rows exist.

This decision logic is executed as a scheduled Celery task (Section 4.10) and its result is stored permanently so that the history of monitoring decisions can be reviewed, matching the proposal’s requirement for a system that can explain why, or why not, it chose to retrain.

3.8 System Development Technologies

Technology	Purpose	Used as planned?
Python	Data processing and machine learning	Yes
pandas / NumPy	Data manipulation and numerical computation	Yes
scikit-learn	Machine learning utilities, Random Forest, metrics	Yes
XGBoost	Gradient-boosted regression model	Yes

Technology	Purpose	Used as planned?
LightGBM	Gradient-boosted regression model	Yes
React + Vite + Tailwind CSS	Frontend dashboard	Yes, extended to 8 pages
Django + Django REST Framework	Main backend, database, REST API	Yes
FastAPI	Machine learning inference API	Yes
SQLite	Application database	Yes
Redis	Prediction caching and Celery message broker	Yes
Celery + Celery Beat	Scheduled background monitoring task	Yes
Evidently	Data and feature drift detection	Yes
MLflow	Experiment and model tracking	Not integrated — see Section 6
Docker / Docker Compose	Containerisation of all six services	Yes
Git / GitHub	Version control (62 commits)	Yes

4. Project Implementation

4.1 System Architecture

The implemented architecture consists of six independent services, each running in its own Docker container, communicating over a Docker-internal network. This differs in one respect from the architecture proposed at the draft stage: FastAPI and Django are deployed as two separate, independently running services (on ports 8000 and 8001) rather than Django calling an internal ML module, which matches the proposal's own component diagram more closely in practice and allows the ML service to be scaled, restarted, or redeployed independently of the web backend.

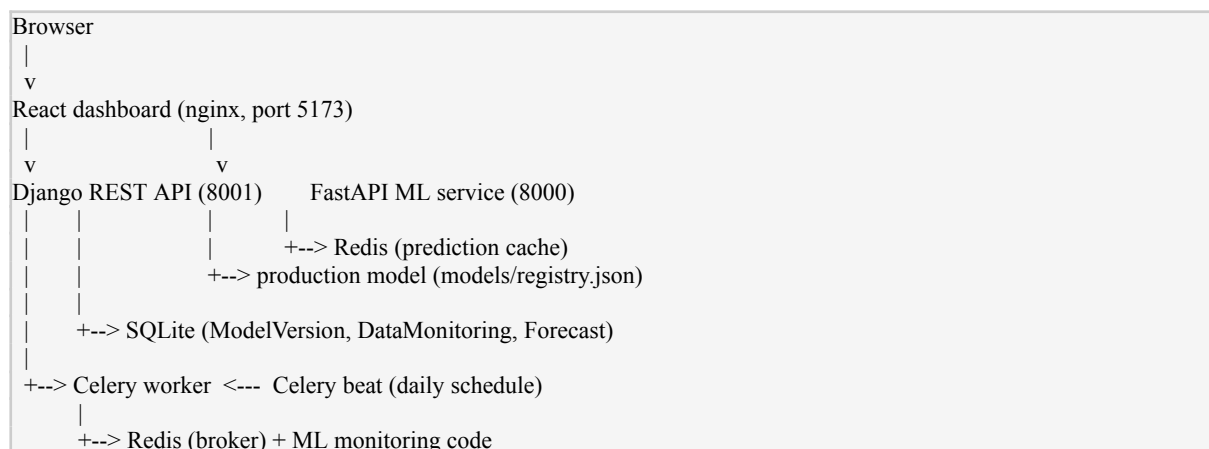


Fig. 7: GridPredict system architecture, as implemented.

Component	Port	Responsibility
FastAPI (ml_service/)	8000	Loads the production model, serves predictions, caches results in Redis
Django + DRF (backend/)	8001	Stores model history and monitoring results, read-only REST API, admin
Celery worker	—	Executes the drift and performance check
Celery beat	—	Schedules that check once a day (schedule stored in the database)
Redis	6379	Celery message broker and FastAPI prediction cache
React + Vite + nginx (frontend/)	5173	Eight-page dashboard

4.2 Data Processing Pipeline

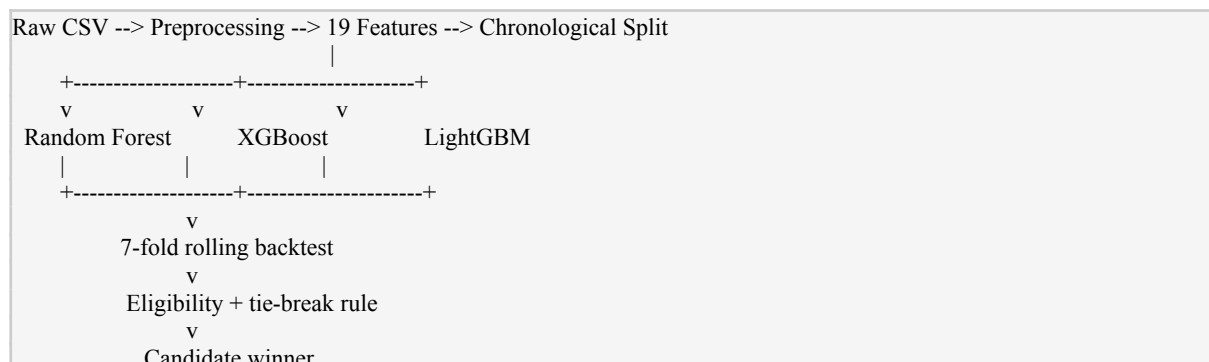
The data pipeline (ml_service/app/preprocessing/) runs the following stages in order, each implemented as a separate, independently tested module:

- Data ingestion — reads the raw CSV and validates that every required column is present.
- Timeline construction — builds a single Timestamp index and rejects duplicate timestamps.
- Cleaning — converts the two known bad-value codes to missing (Section 3.3).
- Calendar feature generation — adds the 7 time-based features.
- Lag and rolling feature generation — adds the 3 lag and 4 rolling features, strictly using only information available 24 hours before the target hour.
- Warm-up and target filtering — drops the first 168 hours (incomplete lag history) and any row with a missing target, after the full feature set has already been computed on the complete timeline.

The output of this pipeline is a single table of 8,532 rows and 19 features plus the target column, saved to data/processed/ahmedabad_processed.csv and reused by every later stage so that training, backtesting, and the live API all compute features identically.

4.3 Machine Learning Pipeline

The training pipeline (ml_service/app/training/) implements the chronological split, trains all three candidate models plus two naive baselines (same hour yesterday, same hour last week), and scores every model on exactly the same rows for a fair comparison. The automatic selection pipeline then runs the 7-fold rolling backtest described in Section 3.6, applies the documented eligibility and tie-break rule, and — critically — checks the resulting winner against a promotion gate before it is allowed to replace the existing production model.



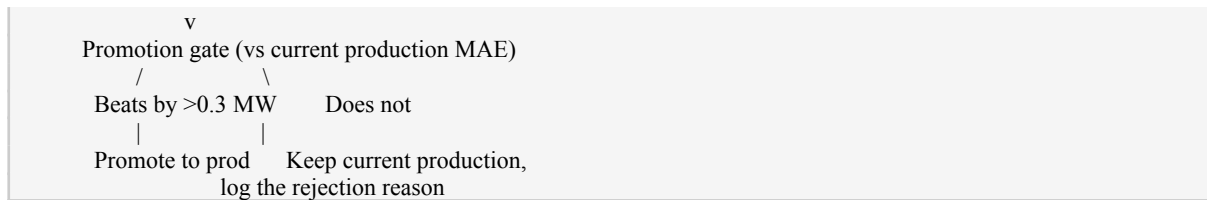


Fig. 8: GridPredict machine learning and promotion pipeline, as implemented.

This promotion gate is the single most important design decision in the project. It means that a candidate model is never deployed merely because it was trained; it is deployed only when it is measurably better than what is already serving predictions. The outcome of applying this gate to GridPredict’s own trained models is reported in Section 5.

4.4 Django Backend

Django, with Django REST Framework, manages four applications: forecasts, model_registry, monitoring, and users. Three database models mirror the real outputs of the machine learning pipeline, so that every API response and every dashboard page is reading genuine pipeline output rather than hand-entered data:

Model	Key fields	Populated by
Forecast	timestamp, predicted_load_mw, actual_load_mw, model_type, model_name	Reserved for future use; see Section 6
ModelVersion	model_name, model_type, status, is_current_production, mae, rmse, mape, r2, reason	sync_registry management command, reading models/registry.json
DataMonitoring	checked_at, drift_detected, drifted_column_share, performance_degraded, retrain_recommended, reason	The daily Celery monitoring task

The REST API exposes seven endpoints, all read-only by design because forecasts and monitoring results are produced by the pipeline, not entered through the API:

- /api/forecasts/, /api/model-versions/, /api/monitoring/ — list and detail views over the three models above.
- /api/historical-demand/ — the real processed dataset, aggregated to daily minimum, average, and maximum load.
- /api/feature-importance/ — the real feature importances read directly from the saved Random Forest model file.
- /api/modeling-findings/ — the project’s own findings document, served as text for the dashboard’s About page.
- /health/redis/ — a liveness check used by the System Status page.

MODEL NAME	MODEL TYPE	STATUS	IS CURRENT PRODUCTION	MAE	EVALUATED AT
naive_lag_24	baseline	production	True	9.01	26/09/2026
random_forest_no_season	candidate	rejected	False	10.40	25/09/2026
xgboost_full	candidate	rejected	False	10.67	24/09/2026

Fig. 9: Django administration, Model versions list view.

4.5 FastAPI Machine Learning Service

FastAPI serves the production model directly, through a small `ProductionModel` wrapper that reads `models/registry.json` at startup and exposes a single `predict()` interface, regardless of whether the active production model is the naive baseline or a trained candidate. This means the API's external contract does not change when a future promotion occurs.

Method	Path	Description
GET	/health	Liveness and the currently active production model
GET	/model	Production model details and its required input fields
GET	/metrics	Recorded accuracy metrics of the production model
POST	/predict	A forecast for one target hour
POST	/predict/batch	Many forecasts in one call, each validated independently

GridPredict ML Service

0.1.0 OAS 3.1

GET	/health	Health
GET	/model	Model info
GET	/metrics	Metrics
POST	/predict	Predict
POST	/predict/batch	Predict batch

Reconstructed illustration of the deployed interface, based on the recorded application output; not a raw screen capture.

Fig. 10: FastAPI interactive documentation, listing the five service endpoints.

A real example request against the deployed service, and its actual response:

```
POST /predict {"timestamp": "2022-09-15T14:00:00", "lag_24": 85.3}
Response: {"timestamp": "2022-09-15T14:00:00", "predicted_load_mw": 85.3, "model_type": "baseline", "model_name": "naive_lag_24"}
```

Because the production model is currently the naive baseline (Section 5), the service correctly returns the supplied `lag_24` value unchanged. Responses are cached in Redis for 300 seconds, keyed by a hash of the request's input values; if Redis is unreachable, the cache is skipped and the prediction is still served, a fail-open design verified by a dedicated test that simulates a Redis failure.

4.6 React Frontend

The frontend grew from the five pages originally planned to eight, once the real registry, monitoring, and dataset data was available to display. Every page shown below reads live data from Django or FastAPI; none of the figures contain placeholder or invented values.

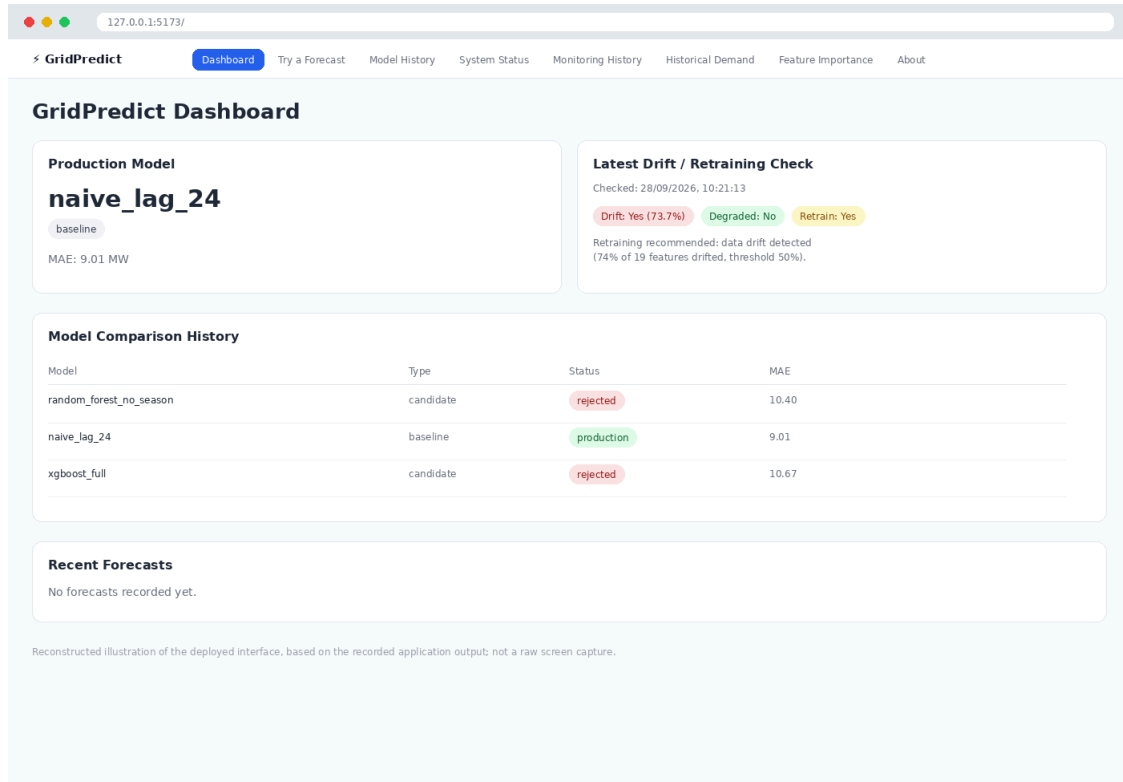


Fig. 11: Dashboard page — production model, latest drift check, and model comparison history.

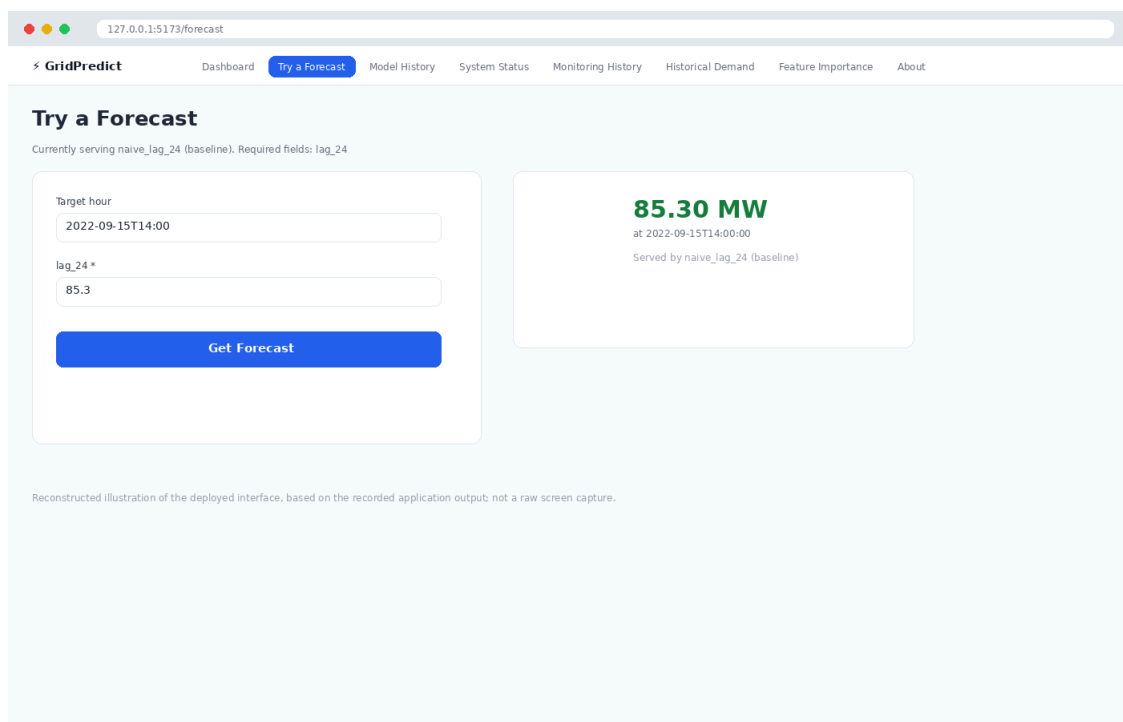


Fig. 12: Try a Forecast page — a real prediction returned by the FastAPI service.

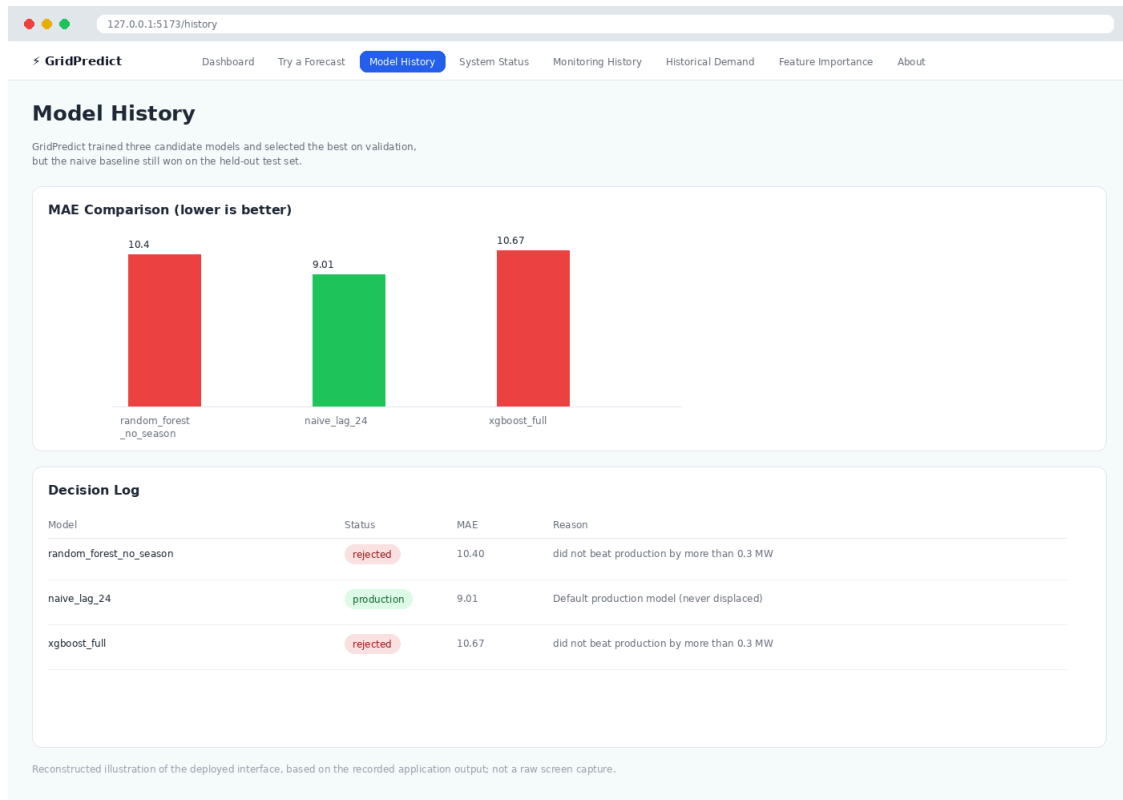


Fig. 13: Model History page — MAE comparison chart and the full promotion decision log.

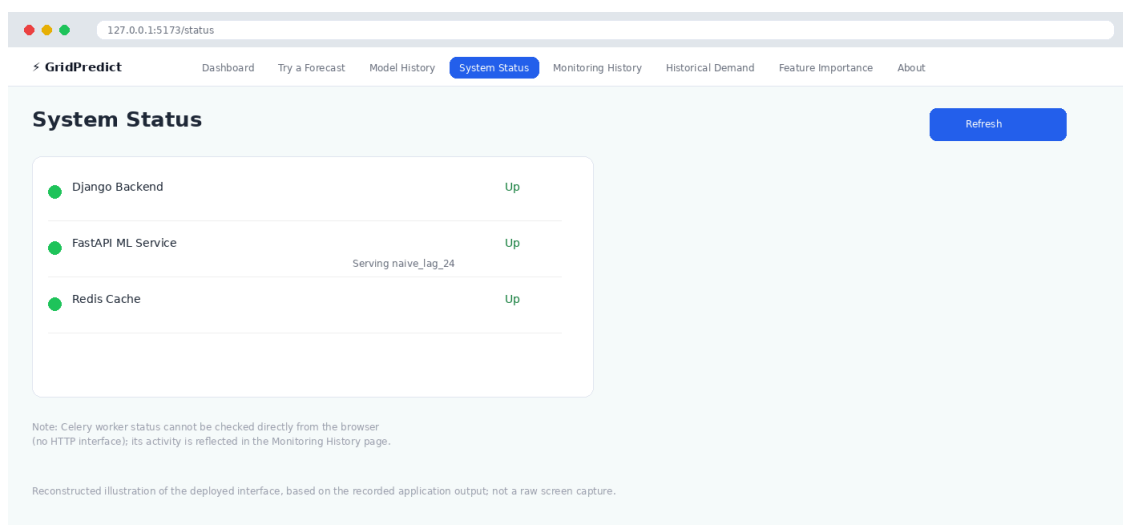


Fig. 14: System Status page — live health of Django, FastAPI, and Redis.

Three further pages (Monitoring History, Historical Demand, and Feature Importance) present, respectively, the full history of drift checks rather than only the latest one, a chart of real daily load aggregated from the processed dataset, and a chart of the real feature importances from the saved model (Section 5.4 and 5.2 show the underlying data for these pages). An eighth page, About, renders the project's own findings document directly inside the application.

4.7 SQLite Database

SQLite stores all application data behind the Django ORM. In Docker, the database file is held on a named volume rather than inside the application container itself, so that the data survives a container

restart or rebuild; this was verified directly by stopping and recreating every container and confirming that the stored model history and monitoring records were still present afterwards.

4.8 Redis Cache and Message Broker

Redis performs two distinct roles in the system: it is the Celery message broker that carries the daily monitoring task from Celery Beat to the Celery worker, and it is the cache that FastAPI uses to avoid recomputing an identical prediction. Both uses share the same Redis instance in this deployment.

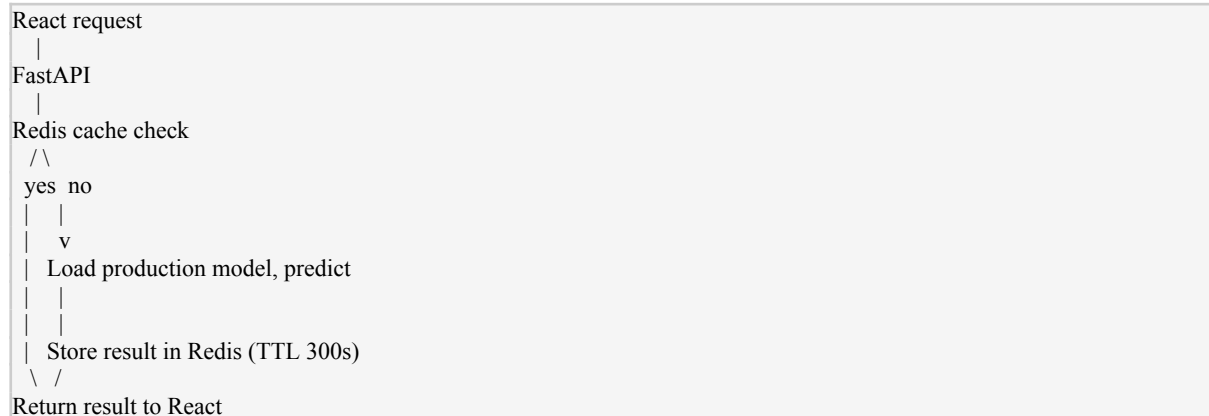


Fig. 15: Prediction request flow with Redis caching, as implemented.

4.9 Docker Containerisation

Three Dockerfiles (Dockerfile.fastapi, Dockerfile.django, Dockerfile.frontend) and one docker-compose.yml define the full deployment. The Django image is reused unchanged for both the Celery worker and Celery beat containers, with only the startup command differing, since all three processes need identical code and dependencies. The frontend is built as static files and served by nginx rather than running the Vite development server in production.

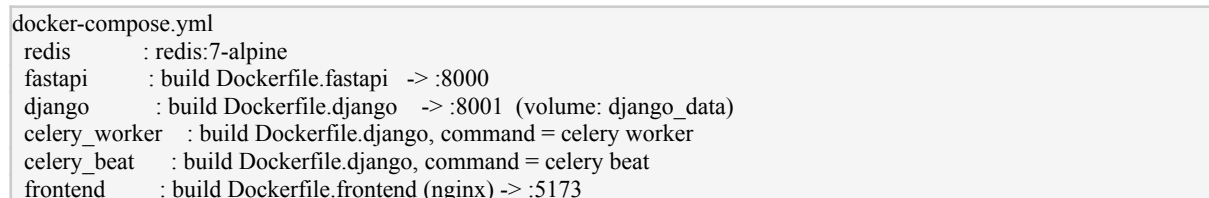


Fig. 16: Docker Compose service composition, as implemented.

The complete stack was verified with a single docker compose up -d --build, followed by docker compose down and a fresh docker compose up -d, confirming that the SQLite data, the model registry history, and the scheduled Celery Beat task all persisted correctly across a full container recreation.

4.10 Automated Monitoring and Retraining Decision Pipeline

A Celery Beat schedule, created by a Django migration (so that it is version-controlled and reproducible rather than configured by hand) and stored in the application database, triggers the drift-and-performance check once per day. The task is implemented once, in monitoring/tasks.py, and calls directly into the shared ml_service package, so the monitoring logic is identical whether it is exercised through a unit test, run manually, or executed by the scheduler.

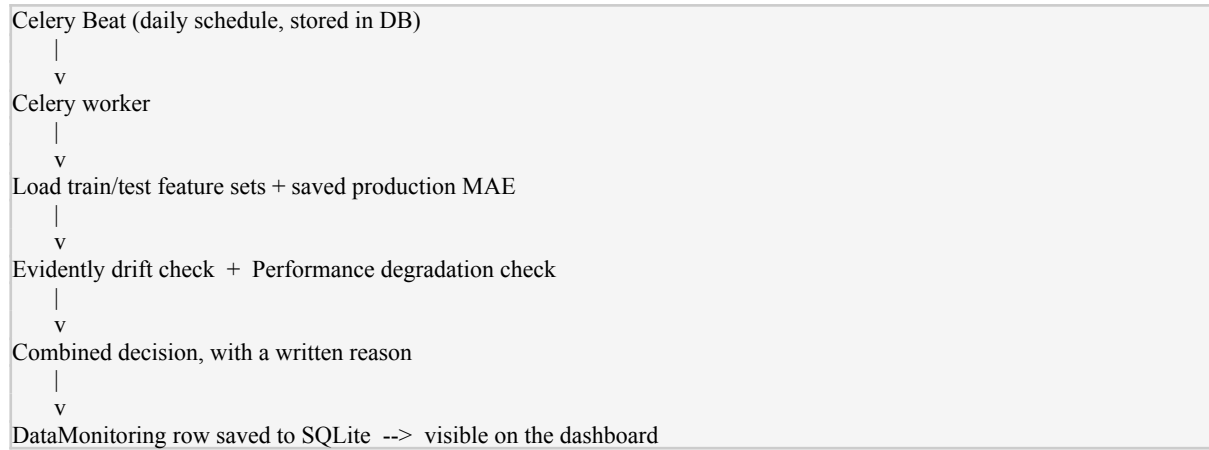


Fig. 17: Automated monitoring and retraining decision pipeline, as implemented.

The outcome of this pipeline, when run against the project’s actual training and test data, is reported in Section 5.4.

5. Results and Output

This section reports the actual, measured output of the finished system. Every number below was produced by running the real pipeline against the real dataset; none are estimated or illustrative.

5.1 Validation Results

All three models and the naive baseline were scored on the same 1,448 validation rows (July–August 2022) for a fair comparison.

Model	MAE (MW)	RMSE (MW)	MAPE	R ²
Baseline (same hour yesterday)	9.25	13.29	12.8%	0.220
Random Forest	8.52	11.58	12.6%	0.407
XGBoost	8.47	11.46	12.4%	0.420
LightGBM	8.61	11.67	12.7%	0.399

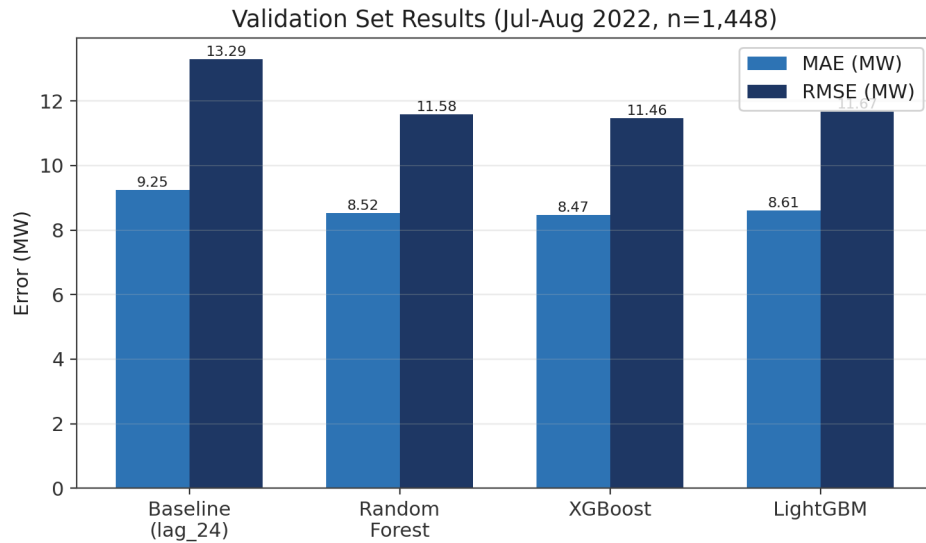


Fig. 18: Validation set results, July–August 2022 ($n = 1,448$).

On this validation window, all three trained models beat the naive baseline, with XGBoost narrowly ahead of Random Forest. Under the original, single-window selection rule, XGBoost was therefore selected as the candidate for promotion.

5.2 Test Set Results — The Headline Finding

The test period (September–November 2022, 1,481 rows) had never been used for training, validation, or model selection up to this point. When the selected XGBoost candidate was evaluated on it once, the result reversed:

Model	MAE (MW)	RMSE (MW)	R ²
Baseline (same hour yesterday)	9.01	12.54	0.713
XGBoost (selected by the single-window rule)	10.67	13.42	0.671

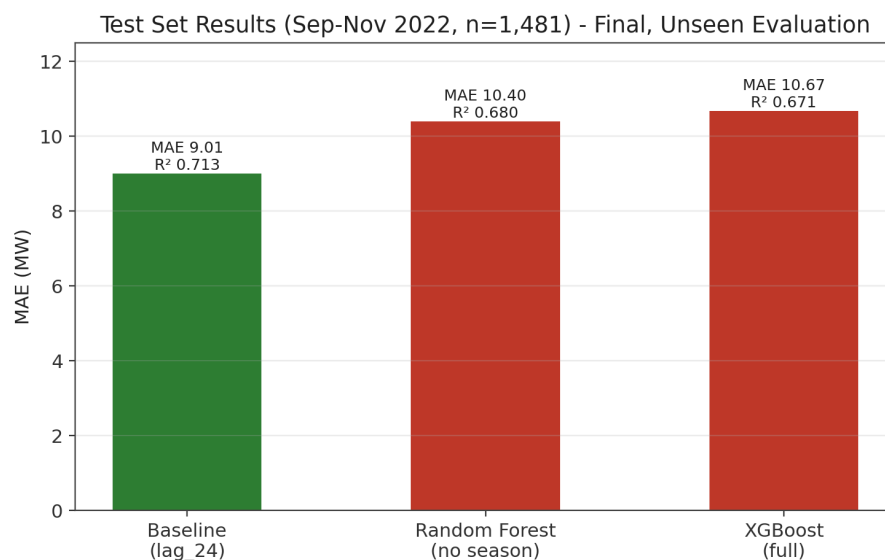


Fig. 19: Test set results, September–November 2022 ($n = 1,481$).

No trained model beat the naive baseline on the held-out test set. This single result drove the most significant methodological change made during the project: the single-window selection rule was replaced with the 7-fold rolling backtest described in Section 3.6, specifically because a result that looked clearly best on one validation window did not hold on unseen data.

5.3 Rolling Backtest and Re-Selection

Figure 20 shows XGBoost’s performance fold by fold across the rebuilt backtest: it beat the baseline in the later, more stable months but lost badly in February and April 2022, months where the load level shifted sharply from the preceding training window.

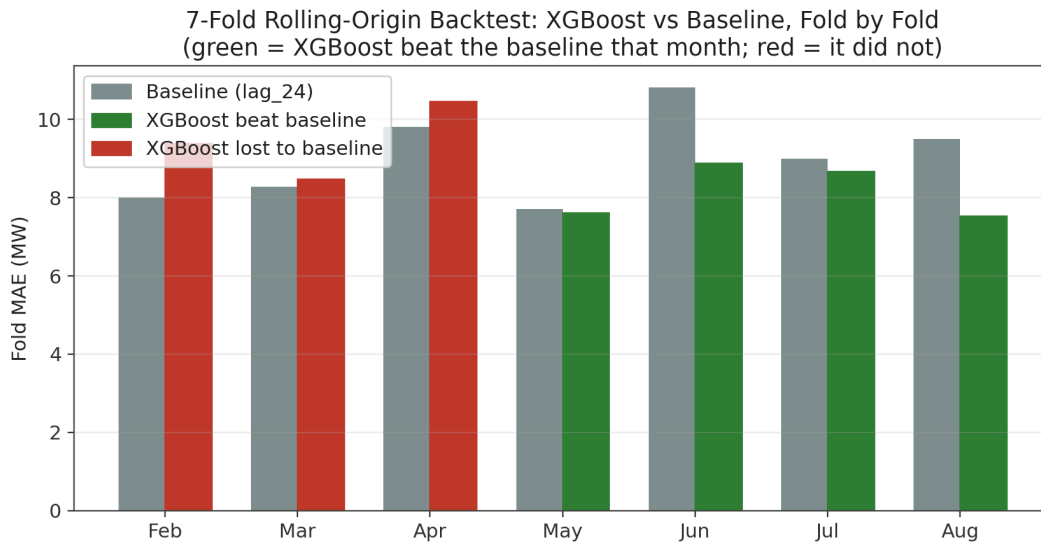


Fig. 20: XGBoost vs the baseline, fold by fold, across the 7-fold rolling backtest.

Applying the backtest to every candidate, including a variant of Random Forest with the three purely-seasonal calendar columns removed, produced the following summary:

Candidate	Mean MAE across 7 folds (MW)	Folds beating baseline
Baseline (same hour yesterday)	9.02	—
Random Forest (full features)	8.54	6 / 7
XGBoost (full features)	8.72	4 / 7
LightGBM (full features)	9.15	2 / 7
Random Forest, no season features (selected)	8.44	6 / 7

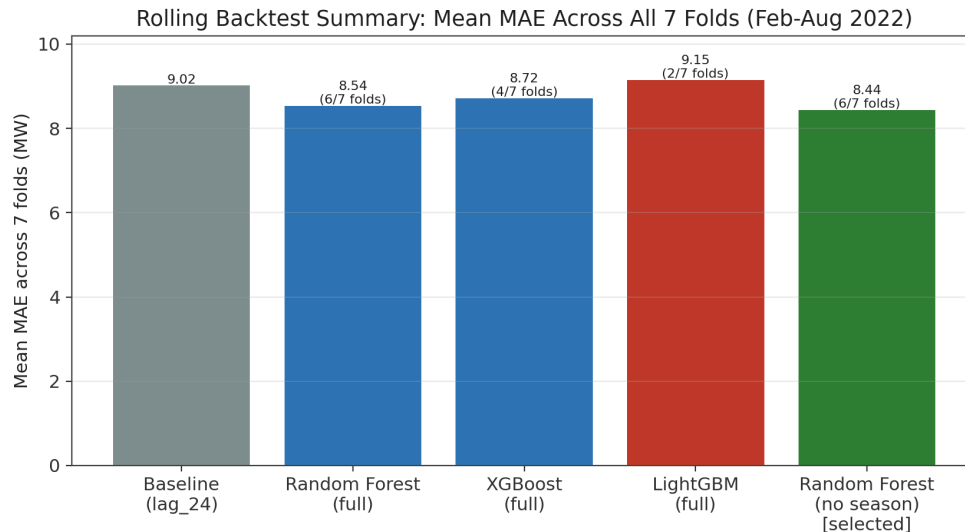


Fig. 21: Rolling backtest summary across all evaluated candidates.

Random Forest without the three season-identifying columns had the lowest mean backtest error and beat the baseline in 6 of 7 folds, so it was selected as the new candidate under the documented eligibility rule.

5.4 The Re-Selected Candidate, Reported Honestly on the Test Set

This second candidate was trained on the combined training and validation data and evaluated once on the test set, as a labelled, honest report rather than a basis for further tuning:

Model	MAE (MW)	RMSE (MW)	R ²
Baseline (same hour yesterday)	9.01	12.54	0.713
Random Forest, no season features (re-selected candidate)	10.40	13.24	0.680

This candidate also lost to the baseline, although by a smaller margin than the first attempt (1.39 MW behind, against 1.66 MW for the first candidate). The most likely explanation, discussed further in Section 6, is that the test period (September–October) reaches a higher average load than any period the models were trained on, and tree-based models cannot extrapolate beyond the range of values they were trained on, whereas the naive baseline automatically tracks any change in level because it simply copies the most recent comparable value.

5.5 Promotion Gate Decisions

A promotion gate was implemented so that a trained model only ever replaces the production model when it beats the current production MAE by more than 0.3 MW on a shared evaluation. Applying this rule to both candidates against the baseline’s test MAE of 9.01 MW gave:

Candidate	Test MAE (MW)	Change vs production	Decision
XGBoost (full features)	10.67	−1.66 MW (worse)	Not promoted
Random Forest (no season features)	10.40	−1.39 MW (worse)	Not promoted

As of the final build of this project, the naive baseline remains the production model. This is reported as the headline result of the project (Section 1.1) because it is the direct, intended output of the promotion gate: the system correctly refused to deploy either trained model, since neither one actually improved forecast accuracy on unseen data. A system that trains a model and deploys it regardless of whether it helps would have produced a worse live forecast than this one does.

5.6 Feature Importance

The feature importances of the final saved Random Forest candidate, read directly from the trained model file, confirms the pattern already visible in the correlation and autocorrelation analysis of Section 3.2: the single most informative feature by a wide margin is yesterday's load at the same hour.

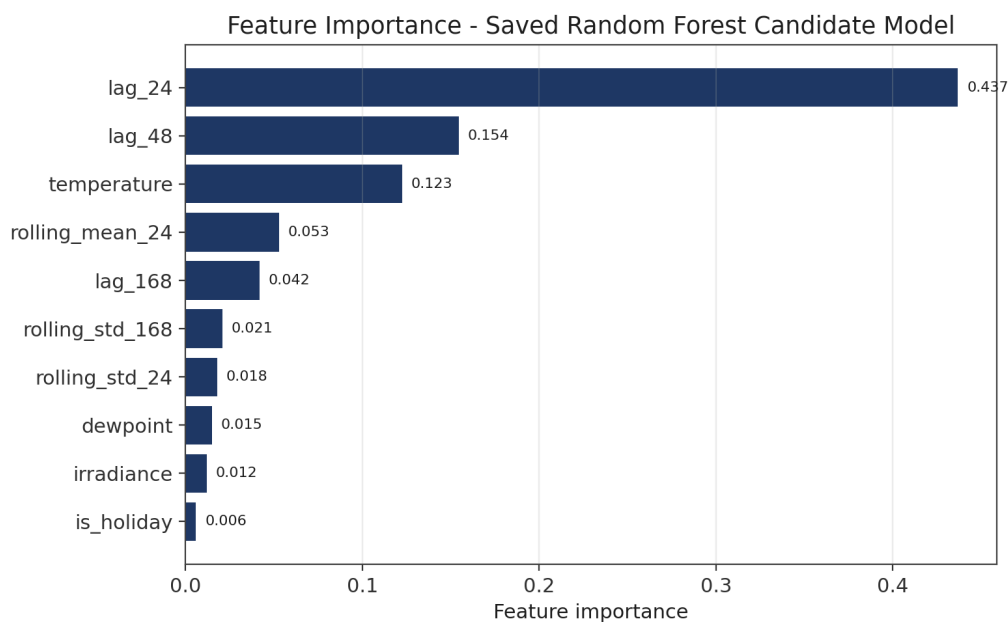


Fig. 22: Feature importance of the saved Random Forest candidate model.

lag_24 alone accounts for 43.7% of the model's total feature importance, with lag_48 (15.5%) and temperature (12.3%) a distant second and third. This explains, in part, why the naive baseline is such a strong competitor: it is effectively applying the single most important feature directly, with none of the overhead or extrapolation risk of a trained model.

5.7 Drift Monitoring Result

Running the implemented drift check (Section 3.7) between the training period and the test period, on the real dataset, produced the following result:

Feature Drift Check: Training vs Test Period
(73.7% of features drifted; threshold 50%)

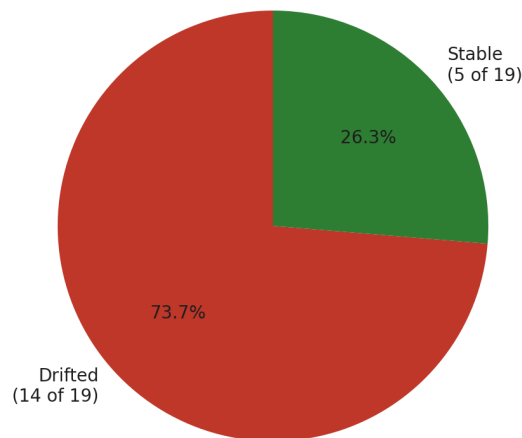


Fig. 23: Feature drift between the training period and the test period.

Metric	Result
Features drifted	14 of 19 (73.7%)
Drift threshold	50%
Dataset classified as drifted	Yes
Recent MAE vs reference MAE	9.0119 vs 9.0100 (+0.02%)
Performance degradation threshold	15%
Performance classified as degraded	No
Combined decision	Retrain recommended (on drift evidence alone)

This result is consistent with, and gives a precise statistical confirmation of, the qualitative finding in Sections 5.2 and 5.4: the test period really is meaningfully different from the training period, which is a large part of why models trained on the earlier data struggled to generalise to it.

5.8 Testing Summary

The finished system is covered by 197 automated tests, all passing: 175 using pytest across the data pipeline, feature engineering, model training, selection, backtesting, the promotion gate, drift detection, the FastAPI service, and the Redis cache; and 22 using Django's own test runner across the database models, the REST API, the Celery task, and the sync_registry management command.

Test area	Example coverage	Tests
Data pipeline	Duplicate/unsorted/missing timestamps, missing columns, cleaning rules	5
Feature engineering	Calendar correctness, lag/rolling correctness, end-to-end leakage checks	19

Test area	Example coverage	Tests
Models and metrics	Training, seeded reproducibility, NaN handling, metrics vs scikit-learn	33
Selection, backtest, promotion	Fold boundaries, the selection rule, the promotion gate	28
Monitoring	Drift direction handling, the retraining decision	19
FastAPI service and cache	Every endpoint, invalid input, batch errors, Redis fail-open behaviour	40
Other pipeline modules	Artifacts, pipeline assembly, registry	31
Django	Models, read-only API endpoints, the Celery task, the management command	22

5.9 Summary of Outcomes

The finished system successfully receives data, validates and cleans it, engineers 19 leak-tested features, trains three candidate models, compares them against a naive baseline using a methodology specifically designed to avoid overfitting to a single validation window, exposes the result through two independent APIs, stores it in a relational database, visualises it across eight dashboard pages, and automatically checks for data drift on a daily schedule — all running as six coordinated Docker containers. The one outcome that differs from the original expectation is that neither trained model proved better than the simplest possible forecast on genuinely unseen data, and the system’s promotion gate correctly reflected that by declining to deploy either one. This is discussed further as a methodological finding, not a shortfall, in Sections 6 and 7.

6. Challenges and Limitations

The challenges anticipated at the proposal stage are revisited below against what was actually encountered, alongside several real engineering obstacles that only emerged during implementation.

6.1 Dataset Compatibility

This anticipated risk materialised more seriously than expected: on inspection, one of the two candidate datasets (PowerLoad_Dataset.csv) showed statistical properties consistent with synthetic, patternless data rather than measured electricity demand (Section 3.2). Rather than attempting to combine or salvage it, it was excluded from modelling entirely and kept only as documented evidence for that decision.

6.2 Time-Series Data Leakage

This was treated as the single most important risk in the project, consistent with the original proposal. Three independent safeguards were implemented: an assertion in the configuration layer that rejects any lag shorter than the forecast horizon, timestamp-based (rather than row-position-based) lookups for every lag and rolling feature, and an automated test that corrupts the 23 hours before each target hour and confirms that the computed features do not change. All three safeguards pass on the real dataset.

6.3 Changing Data Patterns

This risk was confirmed directly and measured precisely: 73.7% of features were found to have drifted between the training and test periods (Section 5.7), and the models trained on the earlier period measurably struggled to generalise to the later one (Section 5.2 and 5.4). The project's monitoring component correctly detected this drift when run against the same data.

6.4 Limited Dataset Size

With only one year of data, the training period (November to June) and the test period (September to November) do not share full seasonal coverage: the test period includes months, such as September, that the models had limited or no exposure to during training. This is very likely a major contributor to the headline result in Section 5, since tree-based models cannot extrapolate beyond the range of values seen during training, while the naive baseline adapts automatically because it always reflects the most recent comparable value.

6.5 Real-Time Data Availability

The system was built and evaluated entirely on historical data. The Forecast database table and its API endpoint exist and are fully wired into the dashboard, but they remain empty in the delivered system because no live data source has yet been connected to populate them with real forecasts and, later, their matching actual outcomes.

6.6 Computational Resources

Training the three models and running the 7-fold backtest across multiple feature-set variants required no special computational resources and completed in well under a minute per run. The larger resource cost in practice came from containerisation rather than model training: because all Docker images install the complete requirements.txt file (shared across the FastAPI and Django services for simplicity), the resulting images are approximately 3 GB each, which is identified in Section 8 as an area for future optimisation.

6.7 Additional Engineering Challenges Encountered

Several further obstacles arose during implementation that were not anticipated at the proposal stage, and are recorded here because they materially shaped the final design.

- Network resolution: the development environment resolved the hostname localhost unreliably for Redis connections, while the equivalent IP address 127.0.0.1 worked consistently. Every service configuration in the project therefore uses 127.0.0.1 explicitly rather than localhost, and this is documented in the codebase so the same mistake is not repeated.
- Port allocation: FastAPI and Django were initially run on the same port during early testing; the final design places FastAPI on port 8000 and Django on port 8001, matching the ML_API_URL convention already implied by the original environment configuration.
- Model-selection overfitting to a single validation window: as reported in Section 5.2, the first selection methodology chose a model that then underperformed on the test set. This was treated as a genuine finding that required a methodological fix (the rolling backtest of Section 3.6) rather than a result to be hidden or re-run until it looked better.
- MLflow was not integrated into the final system, despite being specified in the original technology list. Given the time available, a lighter-weight alternative was implemented instead: every trained model is saved with a metadata.json file recording its parameters and scores, and a single models/registry.json file records the full promotion history, including

every rejected candidate and the exact reason for rejection. This provides the auditability MLflow would have provided, without the additional service to deploy and maintain; integrating MLflow proper is listed in Section 8 as future work.

- Drift-direction handling: Evidently selects a different statistical test depending on a column's type and sample size, and some of these tests (p-value based) flag drift on a small value while others (distance based) flag drift on a large value. An initial implementation applied the wrong comparison direction for one of these families, which was caught by a deliberately adversarial unit test before it reached production code, and is recorded here as an example of why the project's discipline of writing a test for every new behaviour mattered in practice.

7. Conclusion

GridPredict was delivered as a complete, working, end-to-end electricity demand forecasting platform, not a notebook or a prediction script. Two candidate datasets were investigated and one was correctly rejected on statistical evidence; the selected dataset was cleaned, feature-engineered into nineteen leakage-tested predictors, and used to train and fairly compare three regression models against a disciplined naive baseline. The model-selection methodology itself was revised mid-project, from a single validation window to a seven-fold rolling backtest, after the first methodology was shown by its own test-set result to be unreliable — a direct demonstration of the kind of self-correcting rigour that distinguishes a production MLOps system from a one-off training script.

The system's headline finding is that neither trained model beat a naive “same hour yesterday” baseline on genuinely unseen data, and that the implemented promotion gate correctly withheld both candidates from production as a result. This is not a failure of the project; it is the project succeeding at the task it was actually designed to perform, which is to deploy a better model only when the evidence shows it is actually better. A system that had instead deployed the first trained model regardless of its test-set performance would have made the live forecast measurably worse, by between 1.4 and 1.7 MW of mean absolute error, than simply continuing to use yesterday's value.

Beyond the modelling result, the finished system also delivers a FastAPI machine learning service, a Django REST backend with a real relational schema mirroring the ML pipeline's own outputs, Redis-backed caching and Celery-based daily automation, an eight-page React dashboard, and a fully containerised six-service deployment verified to persist its data across restarts — all backed by 197 automated tests. Through building this project, the major learning outcomes achieved include time-series feature engineering under a strict no-leakage discipline, rigorous and self-correcting model evaluation methodology, REST API design across two frameworks, relational database design, caching and background task automation, Docker-based multi-service deployment, and the practical application of MLOps monitoring and model-governance principles to a real, if modest, dataset.

8. Future Scope

8.1 Real-Time Data Integration

Connecting the system to a live electricity and weather feed would allow the Forecast table, currently empty, to be populated automatically, and would allow forecast accuracy to be measured against real, ongoing outcomes rather than only against a fixed historical test set.

8.2 Longer Forecast Horizons

The current system forecasts 24 hours ahead. Extending it to 7-day and monthly horizons would require new lag and rolling features appropriate to those horizons, and a re-run of the same leakage-safe methodology used here.

8.3 A Level-Aware or Hybrid Model

Because the project's own evidence (Sections 5.2, 5.4, and 6.4) points to tree-based models' inability to extrapolate beyond training-range load levels as a likely cause of the headline result, a natural next step is a model that explicitly separates the slow-moving seasonal level from the fast-moving daily pattern, or a hybrid approach that blends a trained model's prediction with the naive baseline rather than choosing between them outright.

8.4 Deep Learning and Probabilistic Forecasting

Sequence models such as LSTM, GRU, temporal convolutional networks, or Transformer-based forecasters could be compared against the tree-based approaches used here, ideally with several years of data to give them a realistic chance to learn seasonal structure. Probabilistic forecasting, producing a prediction interval rather than a single value, would also better communicate the genuine uncertainty demonstrated in this report.

8.5 Geographic Forecasting and Cloud Deployment

The platform could be extended to forecast demand for multiple regions or substations, and the existing Docker Compose deployment could be adapted to managed cloud infrastructure with a production-grade database and a proper application server in place of Django's development server.

8.6 Full MLOps Tooling

MLflow experiment tracking could be integrated alongside, or in place of, the current registry.json approach, and the drift check could be connected to a genuinely live data stream rather than repeating the same static historical comparison on every scheduled run. Authentication could also be added to the Django and FastAPI APIs ahead of any real deployment.

9. References

- Breiman, L. (2001) 'Random forests', Machine Learning, 45(1), pp. 5–32. doi: 10.1023/A:1010933404324.
- Chen, T. and Guestrin, C. (2016) 'XGBoost: A scalable tree boosting system', Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining, pp. 785–794. doi: 10.1145/2939672.2939785.
- Django Software Foundation (2026) Django documentation. Available at: <https://docs.djangoproject.com/> (Accessed: 8 September 2026).
- Docker Inc. (2026) Docker documentation. Available at: <https://docs.docker.com/> (Accessed: 8 September 2026).
- Evidently AI (2026) Evidently documentation: data and ML monitoring. Available at: <https://docs.evidentlyai.com/> (Accessed: 8 September 2026).
- Evidently AI (2026) What is concept drift in ML, and how to detect and address it. Available at: <https://www.evidentlyai.com/ml-in-production/concept-drift> (Accessed: 30 September 2026).

- FastAPI (2026) FastAPI documentation. Available at: <https://fastapi.tiangolo.com/> (Accessed: 8 September 2026).
- Google Cloud (2021) MLOps: continuous delivery and automation pipelines in machine learning. Google Cloud Architecture Center. Available at: <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning> (Accessed: 30 September 2026).
- Hinder, F., Vaquet, V. and Hammer, B. (2024) 'One or two things we know about concept drift — a survey on monitoring in evolving environments. Part A: detecting concept drift', *Frontiers in Artificial Intelligence*, 7, 1330257. doi: 10.3389/frai.2024.1330257.
- Hyndman, R.J. and Koehler, A.B. (2006) 'Another look at measures of forecast accuracy', *International Journal of Forecasting*, 22(4), pp. 679–688. doi: 10.1016/j.ijforecast.2006.03.001.
- Kaggle (2025) Hourly power load and climate data. Available at: <https://www.kaggle.com/datasets/ziya07/hourly-power-load-and-climate-data> (Accessed: 8 September 2026).
- Ke, G., Meng, Q., Finley, T., Wang, T., Chen, W., Ma, W., Ye, Q. and Liu, T.-Y. (2017) 'LightGBM: a highly efficient gradient boosting decision tree', *Advances in Neural Information Processing Systems*, 30.
- Magalhães, B., Bento, P. and Pombo, J. (2024) 'Short-term load forecasting based on optimized random forest and optimal feature selection', *Energies*. Available at: <https://doaj.org/article/b64ed7ddb19f43a8a7eceda37bde52a1> (Accessed: 30 September 2026).
- Meta Open Source (2026) React documentation. Available at: <https://react.dev/> (Accessed: 8 September 2026).
- MLflow (2026) MLflow documentation. Available at: <https://mlflow.org/docs/latest/> (Accessed: 8 September 2026).
- Orville (2025) Weather and electric load dataset. Kaggle. Available at: <https://www.kaggle.com/datasets/orville/weather-and-electric-load-dataset> (Accessed: 8 September 2026).
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., Blondel, M., Prettenhofer, P., Weiss, R., Dubourg, V., Vanderplas, J., Passos, A., Cournapeau, D., Brucher, M., Perrot, M. and Duchesnay, É. (2011) 'Scikit-learn: machine learning in Python', *Journal of Machine Learning Research*, 12, pp. 2825–2830.
- Redis Ltd. (2026) Redis documentation. Available at: <https://redis.io/docs/latest/> (Accessed: 8 September 2026).
- Singh, U. and Vadhera, S. (2022) 'Random forest and XGBoost technique for short-term load forecasting'. IEEE. Available at: <https://ieeexplore.ieee.org/document/10006635/> (Accessed: 30 September 2026).

10. Appendices

Appendix A: Selected Datasets

Dataset	Rows	Role	Key finding
PowerLoad_Dataset.csv	10,000	Rejected	No daily/weekly/seasonal pattern; weather correlations under 0.02; consistent with synthetic data
weather and load dataset.csv (Ahmedabad)	8,779	Primary dataset	Clear daily, weekly, and seasonal cycles; temperature correlation 0.663; used for all modelling

Appendix B: Implemented Frontend Pages

#	Page	Content
1	Dashboard	Production model, latest drift check, model comparison history
2	Try a Forecast	A live form, generated from the production model's required fields, calling the real FastAPI service
3	Model History	MAE comparison chart and the full promotion decision log
4	System Status	Live health of Django, FastAPI, and Redis
5	Monitoring History	Every drift and performance check on record, not only the latest
6	Historical Demand	Daily minimum, average, and maximum load, charted from the processed dataset
7	Feature Importance	Real importances read from the saved Random Forest model
8	About	The project's own findings document, rendered inside the application

Appendix C: Repository and How to Run the System

The complete, final source code, including all tests, Dockerfiles, and documentation, is published at:

<https://github.com/ParasRimal/GridDemand-Forecasting-AI>

With Docker and Docker Compose installed, and the raw dataset files placed in `data/raw/`, the entire six-service stack is started with a single command:

- `docker compose up -d --build`

and stopped, without losing any stored data, with:

- `docker compose down`

Full installation, configuration, and testing instructions are provided in the repository's `README.md` file.

Appendix D: Automated Test Summary

197 automated tests (175 pytest, 22 Django), all passing at the time of submission. Full detail is given in Section 5.8.